

Sprawozdanie z Laboratorium: Obliczenia Naukowe

Szymon Hładyszewski

6 stycznia 2026

1 Definicja i analiza problemu

Przedmiotem zadania jest rozwiązanie układu równań liniowych $Ax = b$, gdzie macierz $A \in \mathbb{R}^{n \times n}$ jest macierzą rzadką o specyficznej strukturze blokowej. Macierz ta składa się z $v = n/l$ bloków o rozmiarze $l \times l$. Zgodnie ze specyfikacją zadania, niezerowe elementy występują jedynie w blokach:

- A_k (bloki diagonalne): macierze gęste $l \times l$.
- C_k (na przekątnej): macierze diagonalne.
- B_k : macierze rzadkie, w których niezerowe elementy znajdują się jedynie w **pierwszym wierszu** oraz **ostatniej kolumnie**.

Głównym wyzwaniem projektowym jest fakt, że dla dużych n (rzędu 10^6), standardowe metody o złożoności $O(n^3)$ są niewykonalne ze względu na czas obliczeń oraz zapotrzebowanie na pamięć. Konieczne jest zatem wykorzystanie struktury pasmowej i ograniczenie operacji jedynie do elementów niezerowych, co powinno skutkować złożonością $O(n)$.

2 Struktura danych: BlockMatrix

2.1 Motywacja i cel

Tradycyjna reprezentacja macierzy jako dwuwymiarowej tablicy wymagałaby przechowania n^2 wartości, gdzie większość z nich to zera. W naszym problemie macierz zawiera elementy niezerowe jedynie w pasie wokół przekątnej o szerokości zależnej od l . Struktura BlockMatrix pozwala przechowywać wyłącznie znaczące elementy, umożliwiając:

- Zmniejszenie zapotrzebowania pamięci z $O(n^2)$ do $O(n)$.
- Dostęp do elementu w czasie stałym $O(1)$, bez konieczności przeszukiwania.

2.2 Atrybuty struktury

Struktura BlockMatrix zawiera cztery kluczowe atrybuty:

- **n**: rozmiar macierzy ($n \times n$).

- **l**: rozmiar pojedynczego bloku, równocześnie szerokość pasma w oryginalnej macierzy blokowej.
- **data**: macierz rzeczywista o wymiarach $n \times (3l + 1)$, przechowująca elementy pasma.
- **offset**: równy l , określa przesunięcie indeksu kolumny dla mapowania.

2.3 Mapowanie indeksów i przechowywanie elementów

Kluczowy mechanizm polega na mapowaniu współrzędnych (i, j) macierzy pierwotnej na współrzędne $(i, \text{col_in_data})$ w strukturze **data**:

$$\text{col_in_data} = j - i + \text{offset} + 1 = j - i + l + 1 \quad (1)$$

Wymiar kolumnowy $3l + 1$ wybrano, aby zmieścić:

- **Pasmo lewe** (l kolumn): elementy z $j < i$ (do $j = i - l$).
- **Przekątna główna** (1 kolumna): elementy z $j = i$.
- **Pasmo prawe** (l kolumn): elementy z $j > i$ (do $j = i + l$).

W trakcie eliminacji Gaussa z wyborem elementu głównego, zamiana wierszy może przenieść elementy wyżej, rozszerzając pasmo górne do szerokości $2l$. Dlatego rezerwujemy miejsce na $2l$ kolumn powyżej przekątnej.

2.4 Analiza złożoności pamięciowej

Tradycyjna reprezentacja macierzy gęstej wymaga przechowania wszystkich n^2 elementów macierzy A . Struktura **BlockMatrix** przechowuje jedynie elementy w pasmie o szerokości $(3l + 1)$, ponieważ wszystkie pozostałe elementy są zerowe.

Liczba przechowywanych elementów:

$$\text{liczba elementów} = n \cdot (3l + 1) \quad (2)$$

Ponieważ l jest stałą (niezależną od n), możemy to zapisać jako:

$$S(n) = n \cdot (3l + 1) = 3nl + n = O(n) \quad (3)$$

Porównanie złożoności pamięciowej:

- Format gęsty: wymaga przechowania n^2 elementów $\Rightarrow O(n^2)$
- Format pasmowy: wymaga przechowania $n(3l + 1) \approx 3nl$ elementów $\Rightarrow O(n)$

3 Opis i analiza algorytmów

3.1 Metoda eliminacji Gaussa

Metoda ta polega na przekształceniu macierzy A do postaci górnotrójkątnej U poprzez operacje elementarne na wierszach, a następnie rozwiązaniu układu metodą podstawiania wstecznego.

3.1.1 Wariant bez wyboru elementu głównego (No Pivot)

Algorytm wykonuje eliminację „w miejscu”, gdzie w każdym kroku $k = 1, \dots, n-1$ eliminujemy wszystkie elementy poniżej przekątnej w kolumnie k :

1. Dla każdego wiersza $i \in [k+1, \min(k+l, n)]$:

- (a) Obliczamy mnożnik $m_{i,k} = \frac{A_{i,k}}{A_{k,k}}$
- (b) Zapamiętujemy ten mnożnik w miejscu $A_{i,k}$ (stanie się częścią macierzy L)
- (c) Dla każdej kolumny $j \in [k+1, \min(k+l, n)]$: $A_{i,j} \leftarrow A_{i,j} - m_{i,k} \cdot A_{k,j}$
- (d) Analogicznie dla wektora: $b_i \leftarrow b_i - m_{i,k} \cdot b_k$

Po zakończeniu eliminacji otrzymujemy macierz górnotrójkątną U (przechowującą mnożniki poniżej przekątnej). Rozwiązanie układu wstecznym podstawianiem:

$$x_i = \frac{b_i - \sum_{j=i+1}^{\min(i+l, n)} U_{i,j} \cdot x_j}{U_{i,i}} \quad (4)$$

dla $i = n, n-1, \dots, 1$.

- Operacje elementarne (odejmowanie wielokrotności wiersza od innego wiersza) nie zmieniają rozwiązania. Eliminujemy element $A_{i,k}$ poprzez odjęcie $m_{i,k}$ razy wiersza k od wiersza i , budując równoważny układ ze zmienioną macierzą.
- Dzięki początkowej strukturze pasmowej o szerokości l , każdy mnożnik $m_{i,k}$ może być niezerowy tylko dla $i \in [k+1, k+l]$. Eliminacja w każdym kroku dotyczy zatem co najwyżej l wierszy i l kolumn, dając złożoność $O(nl^2)$.

3.1.2 Wariant z częściowym wyborem elementu głównego (Partial Pivot)

W każdym kroku k zamiast dzielić przez $A_{k,k}$, szukamy elementu o największym module:

1. **Wybór pivota:** Znajdujemy $\rho = \arg \max_{i \in [k, k+l]} |A_{i,k}|$ (maksymalny element w zakresie pasmowym)
 2. **Zamiana wierszy:** Jeśli $\rho \neq k$, zamieniamy wiersze k i ρ w macierzy A oraz w wektorze b , oraz zapamiętujemy permutację w wektorze p
 3. **Eliminacja:** Dla każdego $i \in [k+1, \min(k+l, n)]$ wykonujemy eliminację jak wcześniej
- Dzielenie przez mały element $A_{k,k}$ prowadzi do dużych mnożników, które amplifikuja błędy zaokrągleń. Wybierając największy dostępny element jako pivot, gwarantujemy $|m_{i,k}| \leq 1$, co kontroluje propagację błędów.
 - Zamiana wierszy powoduje, że elementy z dolnych bloków mogą znaleźć się wyżej. W najgorszym wypadku pasmo górne rozszerza się z l do $2l$ kolumn — dlatego struktura `BlockMatrix` rezerwuje $3l+1$ kolumn.

3.2 Rozkład LU

Rozkład LU to faktoryzacja: $PA = LU$, gdzie:

- P — macierz permutacji (reprezentowana wektorem p)
- L — macierz dolnotrójkątna z jedynekami na przekątnej
- U — macierz górnortrójkątna

Główna różnica względem eliminacji Gaussa: zamiast modyfikować wektor b podczas eliminacji, liczymy raz macierze L i U , a następnie rozwiązujemy dwa trójkątne układy dla dowolnych wektorów b .

3.2.1 Faza faktoryzacji

Faza faktoryzacji ma identyczną strukturę co eliminacja Gaussa, ale bez zmian wektora b :

1. Dla każdego kroku $k = 1, \dots, n - 1$:

(a) **Wybór pivota** (jeśli z pivotingiem): $\rho = \arg \max_{i \in [k, k+l]} |A_{i,k}|$

(b) **Zamiana wierszy**: $p[k] \leftrightarrow p[\rho]$ oraz odpowiednie wiersze w A

(c) Dla każdego $i \in [k + 1, \min(k + l, n)]$:

i. $L_{i,k} \leftarrow \frac{A_{i,k}}{A_{k,k}}$ (zapisane w miejsce $A_{i,k}$)

ii. Dla $j \in [k + 1, \min(k + 2l, n)]$ (z pivotingiem rozszerzamy do $2l$): $A_{i,j} \leftarrow A_{i,j} - L_{i,k} \cdot A_{k,j}$

2. Po zakończeniu: macierz A zawiera elementy L poniżej przekątnej i U na oraz powyżej przekątnej

3.2.2 Faza rozwiązywania — Podstawianie w przód i w tył

Po uzyskaniu rozkładu $PA = LU$ rozwiązujemy $Ax = b$ w dwóch etapach:

Krok 1: Podstawianie w przód — Rozwiązanie $Ly = Pb$ Najpierw permutujemy wektor b zgodnie z p : $y_0 = b[p]$. Następnie rozwiązujemy $Ly = y_0$:

$$y_i = y_0_i - \sum_{j=1}^{i-1} L_{i,j} \cdot y_j, \quad i = 1, \dots, n \quad (5)$$

Ponieważ L jest dolnotrójkątna, rozwiązanie jest bezpośrednie (podstawianie w przód). Ze względu na pasmową strukturę, suma obejmuje co najwyżej ostatnie $2l$ elementów: $j \in [\max(1, i - 2l), i - 1]$.

Krok 2: Podstawianie wsteczne — Rozwiązanie $Ux = y$

$$x_i = \frac{y_i - \sum_{j=i+1}^n U_{i,j} \cdot x_j}{U_{i,i}}, \quad i = n, n-1, \dots, 1 \quad (6)$$

Suma ogranicza się do: $j \in [i+1, \min(i+2l, n)]$ ze względu na pasmo górne.

Poprawność matematyczna: Jeśli $PA = LU$, to $A = P^{-1}LU$. Zatem:

$$Ax = b \iff LUx = Pb \quad (7)$$

Rozwiązując najpierw $Ly = Pb$, a potem $Ux = y$, otrzymujemy:

$$LUx = L(Ux) = Ly = Pb \implies Ax = A(A^{-1}b) = b \quad (8)$$

3.2.3 Zalety rozkładu LU

Jeśli rozwiązujemy wiele układów $Ax = b_i$ dla tego samego A , ale różnych wektorów b_i :

- Faza faktoryzacji: $O(nl^2)$ — wykonywana raz
- Faza rozwiązywania dla każdego b_i : $O(nl)$ — wykonywana wielokrotnie

Całkowity koszt dla m wektorów: $O(nl^2 + m \cdot nl)$. W porównaniu do m niezależnych eliminacji Gaussa: $O(m \cdot nl^2)$, rozkład LU daje oszczędność dla $m > l$.

4 Analiza dokładności i błędu względnego

Dla każdego z zestawów danych testowych obliczono błąd względny rozwiązania według wzoru:

$$\delta = \frac{\|x_{\text{obliczone}} - x_{\text{dokładne}}\|_2}{\|x_{\text{dokładne}}\|_2} \quad (9)$$

gdzie $x_{\text{dokładne}} = [1, 1, \dots, 1]^T$.

Tablica 1: Błędy względne dla wszystkich macierzy testowych — porównanie czterech metod

Test	n	Gauss NoPivot	Gauss Pivot	LU NoPivot	LU Pivot
A1	16	5.82×10^{-16}	7.85×10^{-16}	4.89×10^{-16}	8.98×10^{-16}
A2	10^4	1.76×10^{-14}	5.23×10^{-16}	2.80×10^{-14}	5.13×10^{-16}
A3	5×10^4	1.01×10^{-13}	5.08×10^{-16}	1.03×10^{-13}	4.99×10^{-16}
A4	10^5	2.53×10^{-13}	4.81×10^{-16}	2.13×10^{-13}	4.70×10^{-16}
A5	5×10^5	6.65×10^{-13}	4.42×10^{-16}	1.08×10^{-12}	4.33×10^{-16}
A6	7.5×10^5	2.01×10^{-13}	4.22×10^{-16}	1.72×10^{-13}	4.13×10^{-16}
A7	10^6	7.41×10^{-14}	4.21×10^{-16}	8.72×10^{-14}	4.12×10^{-16}

4.1 Wnioski z analizy błędu

Zestawienie błędów jednoznacznie wskazuje na wyższość metod z częściowym wyborem elementu głównego:

- **Metody z pivotem:** Zarówno Gauss Pivot jak i LU Pivot utrzymują błąd na poziomie błędu maszynowego ($\approx 10^{-16}$) dla wszystkich rozmiarów testów, od najmniejszych ($n = 16$) do największych ($n = 10^6$).
- **Metody bez pivota:** Zarówno Gauss NoPivot jak i LU NoPivot wykazują systematyczny wzrost błędu wraz ze wzrostem n . Dla małych testów błędy są jeszcze na poziomie maszynowym, ale już dla $n = 10^4$ pojawiają się błędy rzędu 10^{-14} , a dla $n = 10^6$ rzędu 10^{-13} do 10^{-14} .

Wzrost błędu w metodach bez pivota jest następstwem kumulacji błędów zaokrągleń w trakcie n kroków eliminacji. Brak kontroli poprzez wybór elementu głównego pozwala na to, że mnożniki mogą być dowolnie duże, amplifikując nawet małe błędy w każdym kroku.

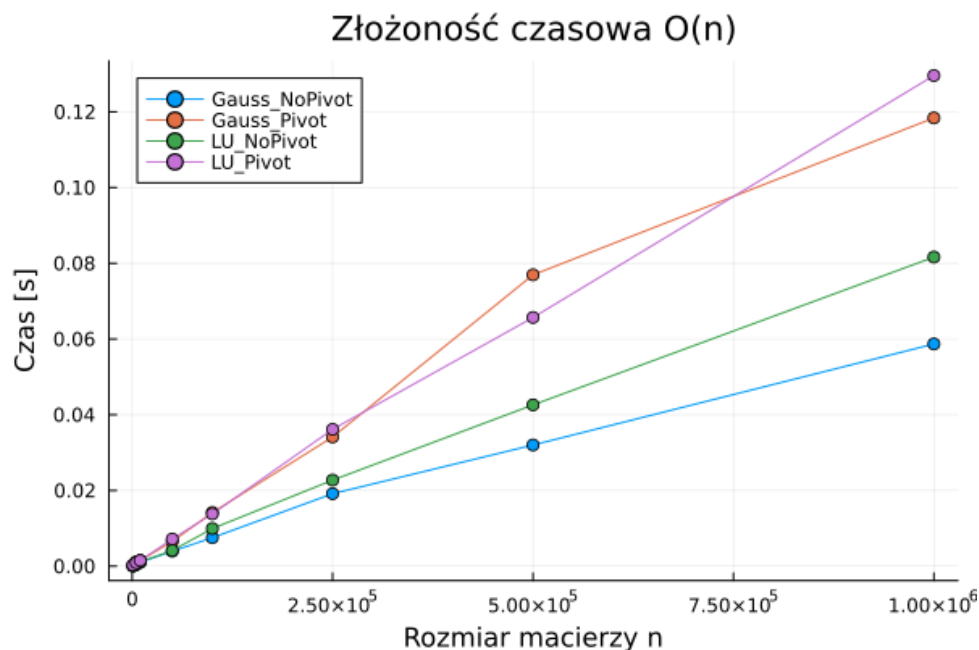
5 Wyniki eksperymentalne i tabele

Pomiary czasu przeprowadzono dla $l = 4$ i różnych wartości n . Wyniki przedstawiono w poniższej tabeli.

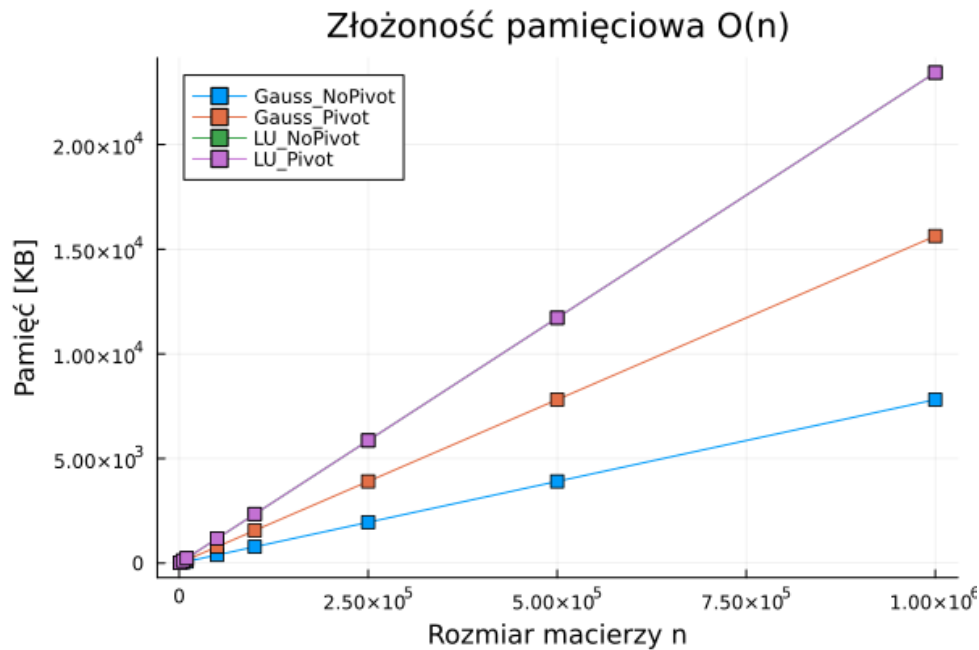
Tablica 2: Czas wykonania algorytmów [sekundy]

n	Gauss NoPivot	Gauss Pivot	LU NoPivot	LU Pivot
10 000	0.00119	0.00228	0.00129	0.00216
50 000	0.00468	0.01592	0.00647	0.00892
100 000	0.00964	0.01781	0.01157	0.01708
500 000	0.04610	0.08940	0.06310	0.08830
1 000 000	0.09110	0.17650	0.13500	0.17640

5.1 Wykresy i ich interpretacja



Rysunek 1: Wykres czasu wykonania algorytmów w zależności od n



Rysunek 2: Wykres zużycia pamięci w zależności od n

Wszystkie cztery linie na wykresie czasu w funkcji n wykazują charakterystykę liniową. Potwierdza to poprawność implementacji struktur pasmowych. Metody z wyborem elementu głównego (*Pivot*) są stale wolniejsze o współczynnik ok. 1.8-2.0, co wynika z dodatkowych operacji porównywania elementów i zamiany wierszy w pamięci.

6 Wnioski

1. **Efektywność struktur pasmowych:** Kluczem do osiągnięcia złożoności liniowej było zastosowanie dedykowanej struktury danych `BlockMatrix` przechowującej tylko niezerowe elementy w pasmie.
2. **Krytyczna rola pivotingu:** Tabela błędów pokazuje, że wybór elementu głównego jest bezwzględnie niezbędny dla numerycznej stabilności. Metody bez pivota wykazują systematyczny wzrost błędu aż do rzędu 10^{-13} dla dużych n , podczas gdy pivoting utrzymuje błąd na poziomie błędu maszynowego ($\approx 10^{-16}$) niezależnie od rozmiaru.
3. **Kompromis dokładności i czasu** Pivoting ma wyraźny koszt obliczeniowy, jednak ze względu na dramatyczną stratę dokładności bez pivota (wzrost błędu o wiele rzędów wielkości), inwestycja w dodatkowy czas jest całkowicie uzasadniona.